Final GridLock Hackathon Pipeline (Locked)

Module 0 — Data Preparation

Input:

Approved parking violation records

Feature Engineering:

hour

day_of_week

month

is_weekend

violations_list

num_violations

max_severity

vehicle_weight

junction_weight

Output:

Engineered violation dataset

Module 1 — Spatial Hotspot Detection

Algorithm

DBSCAN

eps = 0.00000962 (~61m)

min_samples = 20

metric = haversine

Output

cluster_id

hotspot centroid

cluster size

noise points

Current result:

506 hotspots

6.17% noise

This becomes the spatial backbone of the entire system.

Module 2 — Temporal Intelligence

For every hotspot:

Compute:

hour distribution

day-of-week distribution

monthly counts

Metrics:

peak activity window

weekday/weekend behavior

active_months

Output:

Hotspot persistence profile

Example:

Cluster 16

Active: 6/6 months

Peak Window: 6–8 AM

Module 3 — Multi-Factor Risk Engine

Inputs

log_violations_norm

avg_severity_norm

avg_vehicle_weight_norm

avg_junction_weight_norm

active_months_norm

avg_num_violations_norm

Formula

```
priority_score = (
  0.45 * log_violations_norm +
  0.20 * avg_severity_norm +
  0.10 * avg_vehicle_weight_norm +
  0.10 * avg_junction_weight_norm +
  0.10 * active_months_norm +
  0.05 * avg_num_violations_norm
)
```

Risk Levels

Percentile-based:

0-50% → Low

50-80% → Medium

80-95% → High

95-100% → Critical

Output:

Risk-ranked hotspot table

Module 4 — Hotspot Trajectory Analysis ⭐

This replaces XGBoost.

Step 1

For every cluster:

Build:

Month

Violation Count

Example:

Nov 80

Dec 92

Jan 110

Feb 125

Mar 140

Step 2

Fit:

LinearRegression()

on:

Month Number

→

Monthly Violations

Store:

slope

for each hotspot.

Step 3

Data-Driven Thresholds

Compute slope distribution across all 506 hotspots.

Use:

75th percentile slope

↓

Escalating

Use:

25th percentile slope

↓

Declining

Everything else:

Stable

No arbitrary thresholds.

Output

Escalating

Stable

Declining

Example:

Cluster 114

Slope = +28.4

Status = Escalating

Module 5 — Anomaly Detection ⭐

Goal:

Find hotspots behaving unusually this month.

Per-Cluster Statistics

For each hotspot:

mean_monthly

std_monthly

computed from its own history.

Z-Score

z_score =

(current_month - cluster_mean)

/

cluster_std

Guard Condition

If:

active_months < 2

or

cluster_std = 0

label:

Insufficient History

No anomaly score.

Threshold

z_score > 1.5

↓

Abnormal Surge

Output:

Watchlist

Example:

Cluster 16

Current Month: 420

Historical Mean: 190

Z-score: 2.1

Status:

Abnormal Surge

Module 6 — Patrol Allocation Optimizer

Combine:

Risk Level

Trajectory

Anomaly Flag

into patrol ranking.

Patrol Priority Logic

Example:

Critical

+

Escalating

+

Abnormal Surge

↓

Highest Priority

Output:

Top 20 patrol zones

Example:

1. Cluster 114

Critical

Escalating

Surge

2. Cluster 16

Critical

Stable

Module 7 — Dashboard

Tab 1

Hotspot Map

DBSCAN clusters

Heatmap

Risk colors

Tab 2

Risk Zones

Low

Medium

High

Critical

rankings.

Tab 3 ⭐

Trajectory View

For selected hotspot:

Show:

Month-by-month line chart

plus:

Slope

Status

Example:

Cluster 114

Nov → Mar trend line

Status:

Escalating

This is the evidence behind the label.

Tab 4

Watchlist

Abnormal Surges

Tab 5

Patrol Recommendations

Priority queue

Deployment windows

Tab 6

Impact Simulator

User chooses:

10%

20%

30%

violation reduction.

System shows:

Projected Risk Reduction

Not traffic-flow reduction.

Final Pitch

> **We do not train a model to reproduce our own labels. Instead, we identify spatial parking hotspots using DBSCAN, quantify operational risk through a multi-factor risk engine, detect hotspots that are worsening over time through trajectory analysis, flag abnormal surges requiring immediate attention, and convert these signals into prioritized patrol recommendations. Every recommendation is traceable to historical violation evidence and is directly actionable for enforcement agencies.**

This version is technically defensible, matches your data limitations, and aligns closely with the hackathon problem statement.